Who controls the failover?



Sovereign global traffic management with Kubernetes

- **TRAFFIC MANAGEMENT**
- **DNS STEERING**
- **MULTI-REGION RESILIENCE**

Yury Tsarev · Creator & Project Lead, k8gb

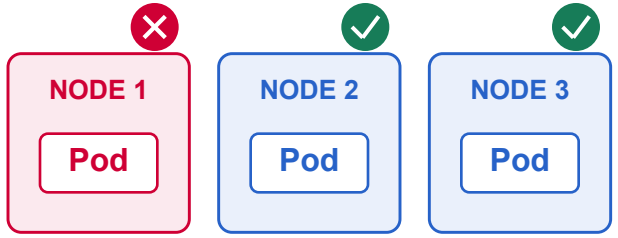
24 October 2026 · Saarbrücken

From node resilience to cluster failover

Spread replicas, then widen the failure boundary.

01 NODES

One Kubernetes cluster

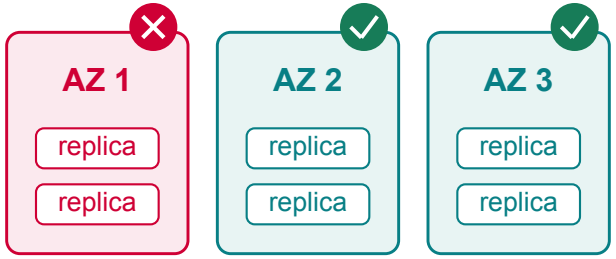


Spread replicas

Survive a worker failure.

02 AVAILABILITY ZONES

One Kubernetes cluster

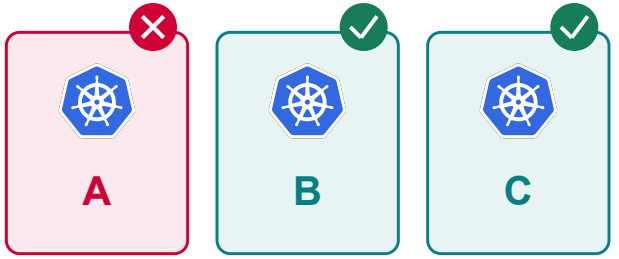


Spread across zones

Survive a zone failure.

03 CLUSTERS

Independent clusters



Steer across clusters

Keep one application hostname.

When an entire cluster is unavailable, who steers the next request?

k8gb in one minute

Two or more clusters. No fixed cluster-count limit in k8gb.



CNCF INCUBATING

Kubernetes-native

Readiness becomes DNS eligibility.

Decentralized

A control loop at each participating cluster.

Declarative

Review traffic steering in Git.

Global application reachability across Kubernetes clusters.

From Absa production to CNCF incubation

An operational need grew into a community project.

PRODUCTION ORIGINS

Born at Absa

Multiple on-prem datacenters, then hybrid deployments

30 MAR 2021

CNCF Sandbox

Contributed to the cloud native community

PUBLIC ADOPTION

Beyond Absa

Millennium bcp
Open Systems
PagBank

18 JUL 2026

CNCF Incubating

A new milestone for the project and its community

Born from a production problem. Built with the community.

Resilience in production

Millennium bcp · Results reported in a CNCF case study

DNS INCIDENT RESPONSE

70%

shorter response times

for DNS-related incidents

5

regions

Two cloud providers
+ on-premises

Zero downtime

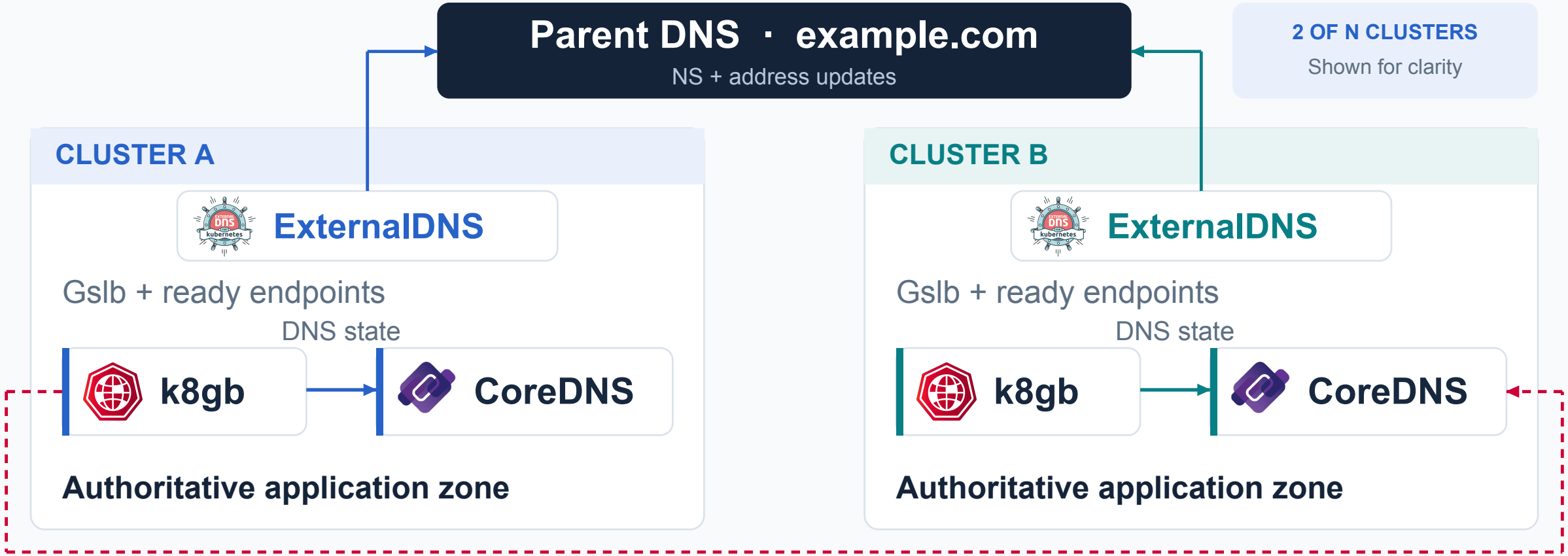
Reported during regional failover tests

CNCF case study · 16 September 2025

cncf.io/case-studies/millennium-bcp

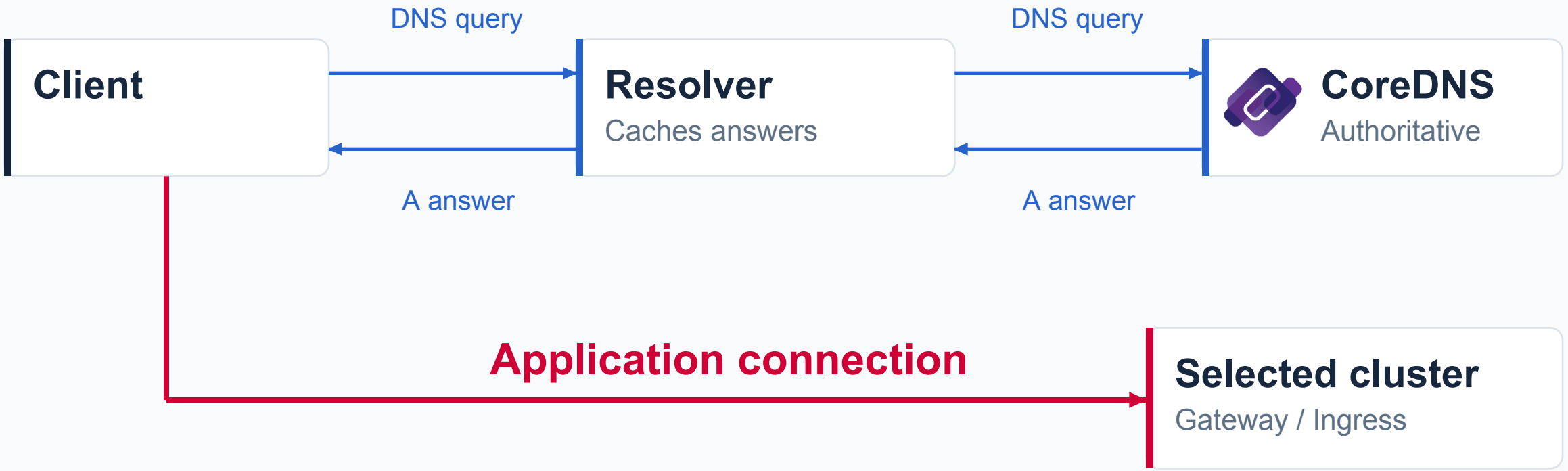
Published results from Millennium bcp’s deployment.

Independent clusters, shared DNS contract



One direction shown; each controller queries peer CoreDNS.

DNS selects an address. The client connects.



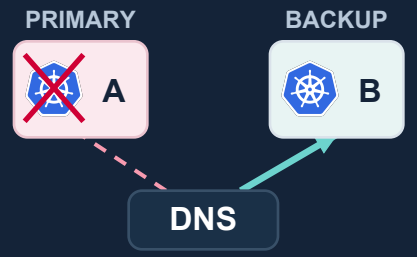
DNS is outside the application data path.

Choose how eligible clusters are used

Illustrative DNS selections across Kubernetes clusters.

FAILOVER

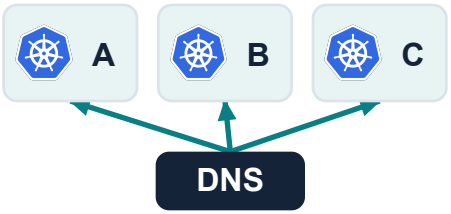
Prefer primary



Return to primary on recovery

ROUND ROBIN

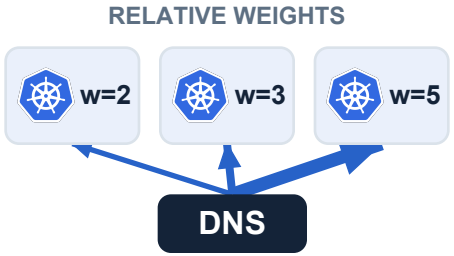
Use healthy clusters



Active / active DNS answers

WEIGHTED ROUND ROBIN

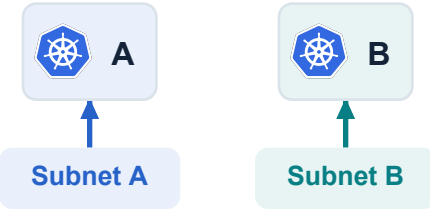
Express preference



Request shares are approximate

GEOIP

Map client subnets



Needs GeoIP data + ECS

All modes change DNS answers, not existing connections.

Traffic steering with a declarative manifest

Gslb is a Kubernetes custom resource.

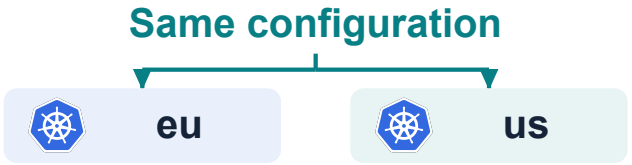
gslb.yaml — KUBERNETES CUSTOM RESOURCE

```yaml
apiVersion: k8gb.io/v1beta1
kind: Gslb
metadata:
  name: test-gslb-failover
  namespace: test-gslb
spec:
  resourceRef:
    apiVersion: gateway.networking.k8s.io/v1
    kind: HTTPRoute
    name: payments
  strategy:
    type: failover
    primaryGeoTag: eu
```

Prefer A · eu

While it has eligible targets.

Deploy per cluster



Each cluster observes health locally.

The route, zone, and matching geotags are prerequisites.

Choose parent DNS. Keep traffic steering.

Managed or self-hosted: the application zone stays with CoreDNS.

Managed DNS

Operated by your provider

Provider API

OR

Self-hosted DNS

Operated by your team · BIND9

RFC 2136 + TSIG

ExternalDNS

Updates parent-zone delegation

SAME TRAFFIC STEERING IN EVERY CLUSTER

Gslb + k8gb → **CoreDNS**

Your choice of parent DNS.

Your Gslb strategy stays the same.

You define what a “region” means

Global traffic management can follow your own topology.

ILLUSTRATIVE clusterGeoTag VALUES



REGION

`eu-west`



DATACENTER

`dc-1`



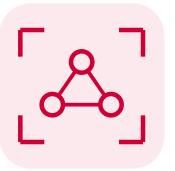
RACK

`rack-a`



SERVER

`server-01`



CUSTOM

`sovereign-a`



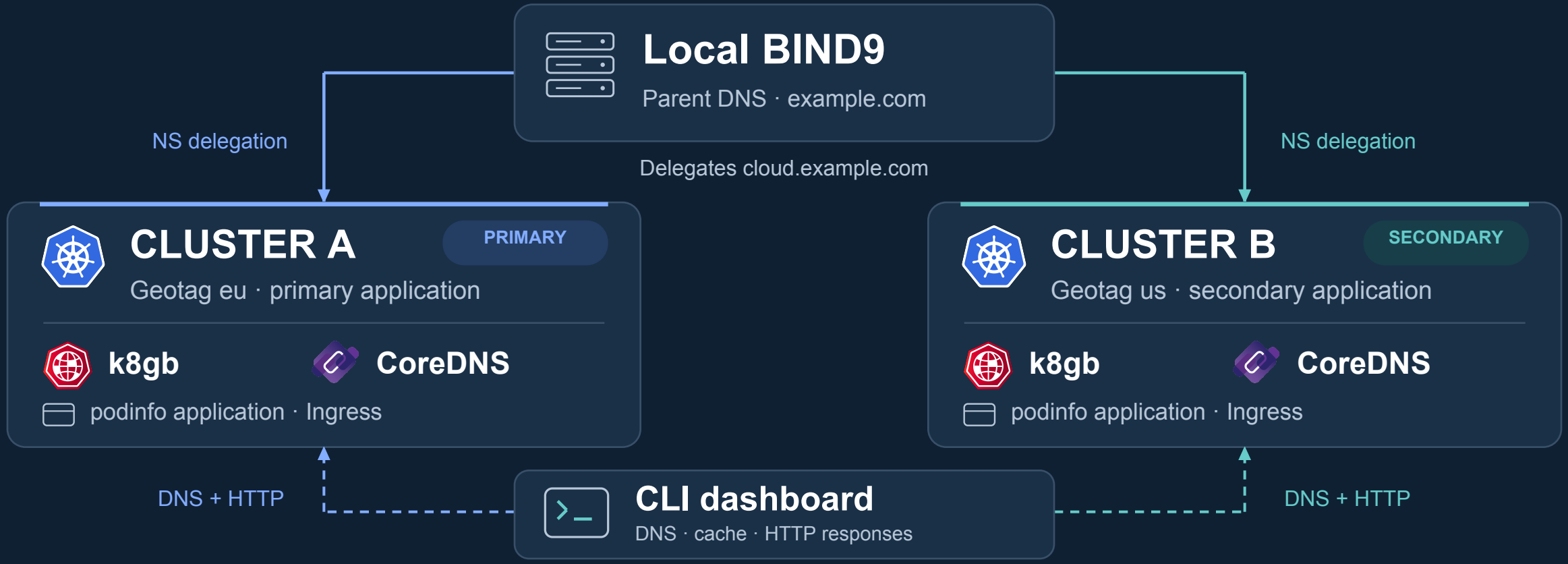
Each tag identifies a k8gb cluster.

Rack or server boundaries use independent clusters, including single-node clusters.

Choose boundaries that match your sovereignty model.

One hostname. Two clusters.

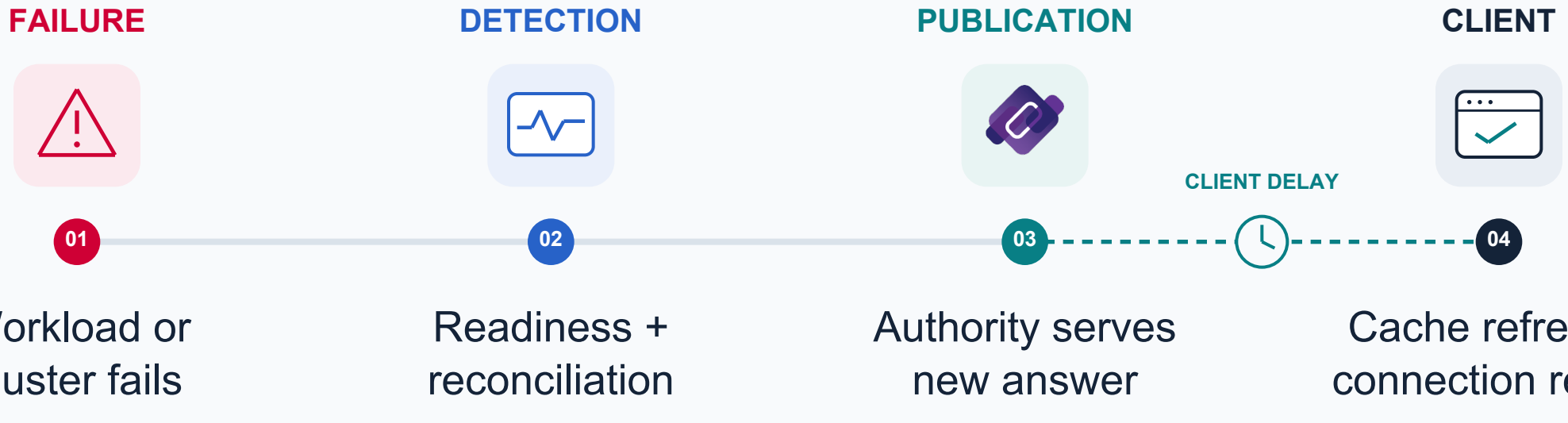
All local. Two application clusters represent different locations.



Observe A → remove A's ready targets → watch B → restore A

A new DNS answer is not an instant switch

Detection, DNS publication and client recovery happen at different times.



TIMING VARIES

Readiness · reconciliation · caches · retries

New DNS answer ≠ recovered clients

Measure recovery at the resolver and application.

Readiness must reflect the real operation

Kubernetes runs Pod probes. k8gb consumes EndpointSlice readiness.



HEALTH ENDPOINT

`GET /healthz`

HTTP 200

The shallow check passes.

REAL REQUEST

Payment cannot be saved

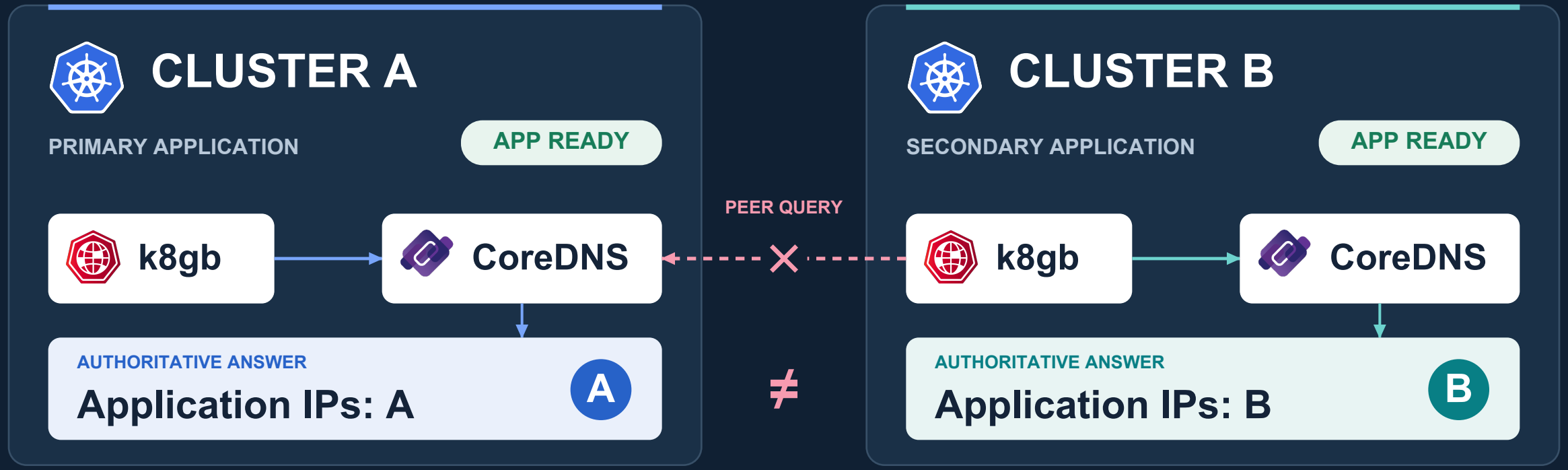


Replication and safe writer ownership belong to the application.

k8gb steers traffic. The application must be ready to serve.

A partition can create two correct-looking answers

Both applications are ready. Clients can still reach both DNS authorities.



B cannot obtain A's peer-health answer.

A failed health query does not prove the application is down.

Compare the answers your users can receive

During a partition, compare the same hostname from each viewpoint.

ILLUSTRATIVE PARTITION

CoreDNS A	Cluster A IPs
CoreDNS B	Cluster B IPs
Resolver cache	May still return A
Real request	Verify the result

Correlate the signals

- Pod readiness
- k8gb errors and logs
- Application results

METRIC EXAMPLE · ONE HOSTNAME

1 cluster → 3 healthy IPs

Healthy-record count: 3

Query both authorities. Check the cache. Test a real request.

Perform end-to-end resilience tests

A is primary, B is healthy. Change one condition at a time.

CHANGE ONE CONDITION	VERIFY DNS + APPLICATION
Remove A’s ready endpoints 01 · WORKLOAD FAILURE	→ **Both DNS answers → B** Verify successful requests to B.
Make cluster A unavailable 02 · CLUSTER FAILURE	→ **B serves DNS and application** Test through a client resolver.
Block B’s peer-health lookup 03 · PEER DNS PARTITION	→ **A answers A · B answers B** Check the impact on real requests.
Restore A’s readiness 04 · RECOVERY	→ **Both DNS answers → A** Check requests and data correctness.

Time DNS convergence and the return of successful requests.

Control traffic steering. Test recovery end to end.



CONFIGURE

Version the Gslb manifest.

VERIFY

Check DNS answers and real requests.

TEST

Automate failover and failback tests.

Test the full path: readiness → DNS → successful requests.

Questions?

How would failover work in your environment?

Explore k8gb



k8gb.io

Yury Tsarev

Creator & Project Lead · k8gb

linkedin.com/in/yurytsarev

SOURCE & CONTRIBUTIONS

github.com/k8gb-io/k8gb

PRODUCTION CASE STUDY

Millennium bcp

cncf.io/case-studies/millennium-bcp

Three components. Three distinct jobs.

k8gb controller

Readiness + strategy
→ eligible targets

CoreDNS

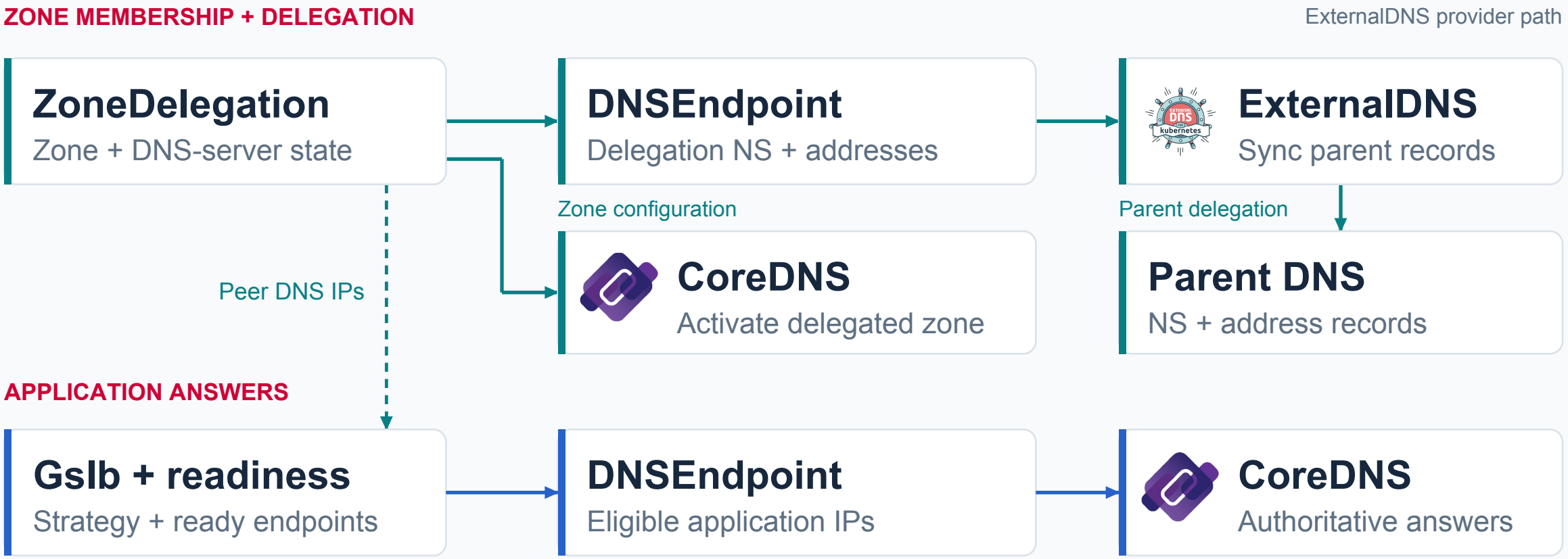
Delegated zone
→ DNS answers

ExternalDNS

Delegation state
→ parent DNS updates

Decide locally. Answer authoritatively. Delegate explicitly.

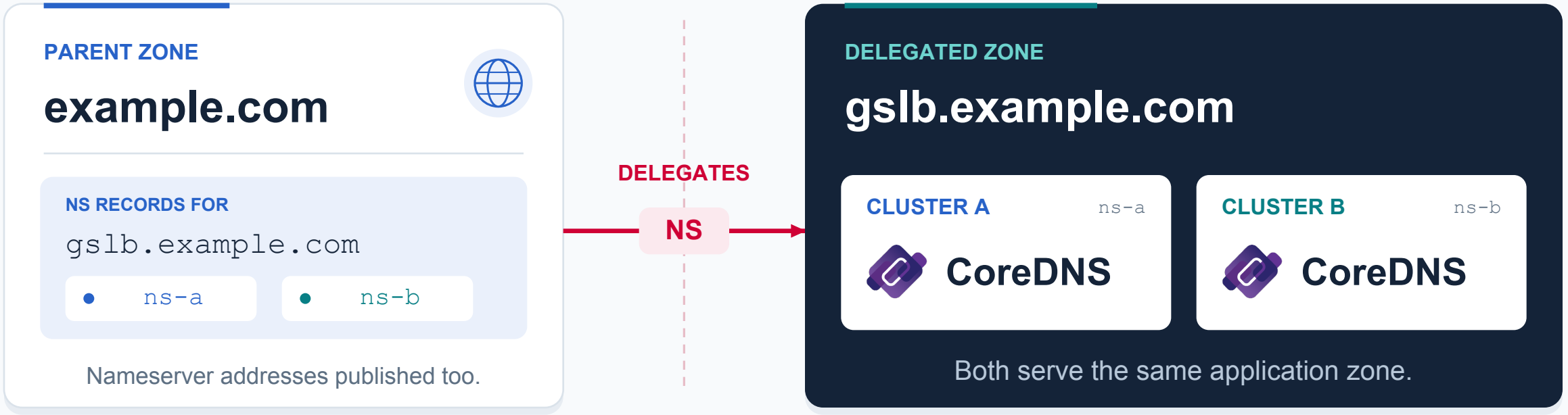
Two k8gb reconciliation loops feed DNS



Membership refresh and application reconciliation run separately.

Delegation is the authority boundary

Parent DNS delegates the zone. CoreDNS answers application lookups.

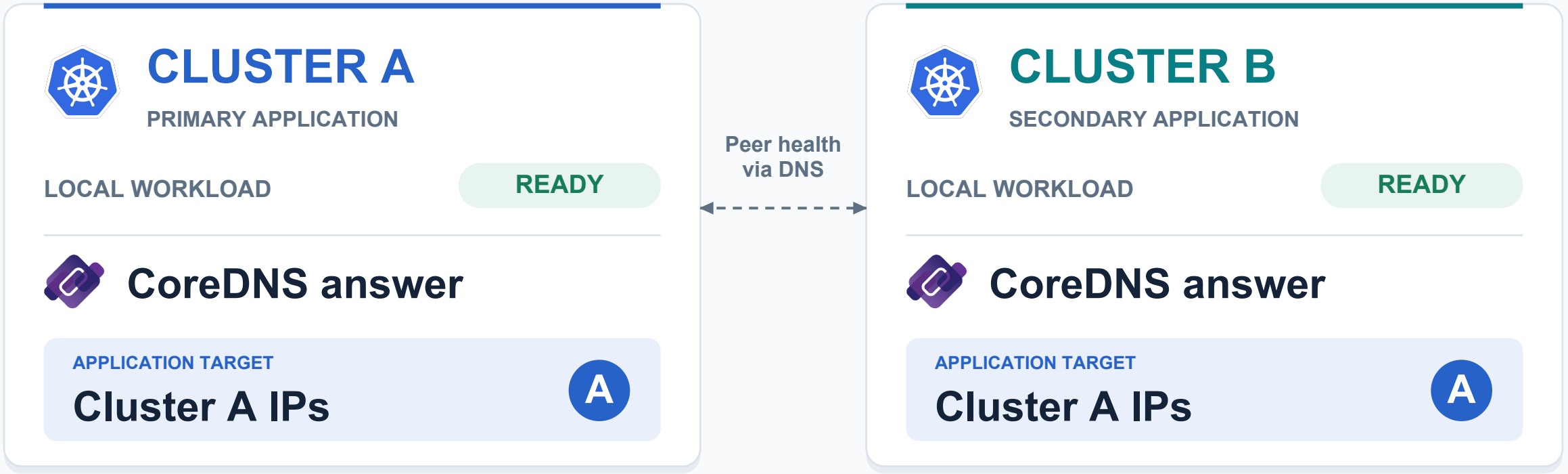


A app.gslb.example.com → Eligible application IPs

Own the name, the delegation, and DNS recovery.

Normal operation: both authorities prefer A

Both applications are ready. Cluster A is the preferred target.

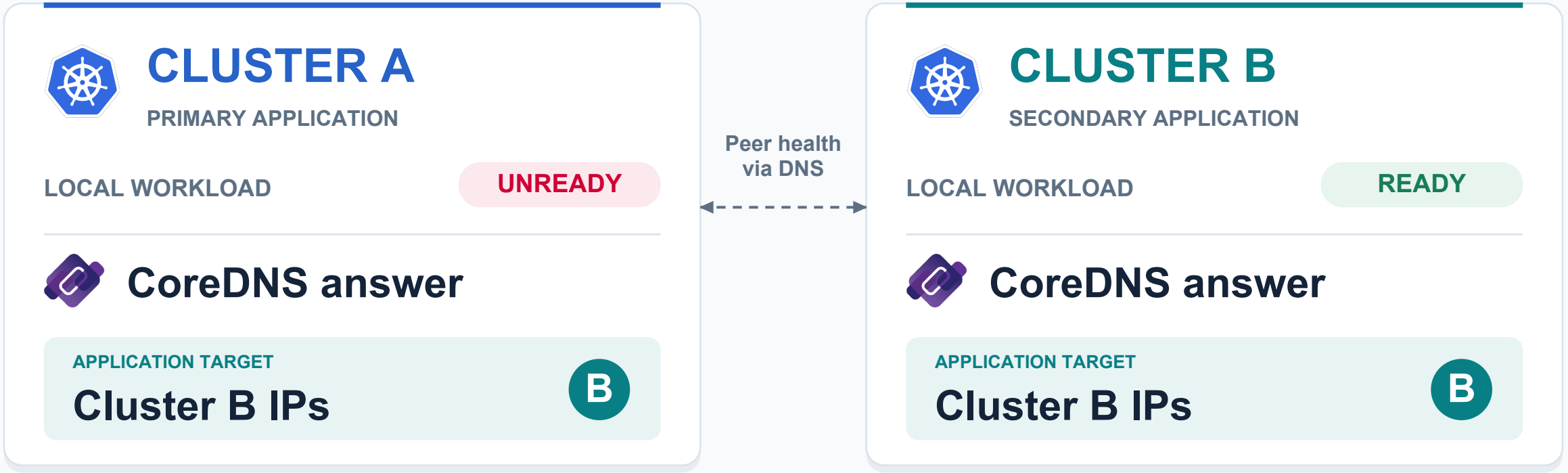


Each k8gb controller queries peer CoreDNS.

Both authorities agree after reconciliation; client caches may lag.

A loses ready targets: publish B

A’s application is unready; its DNS authority is still available.

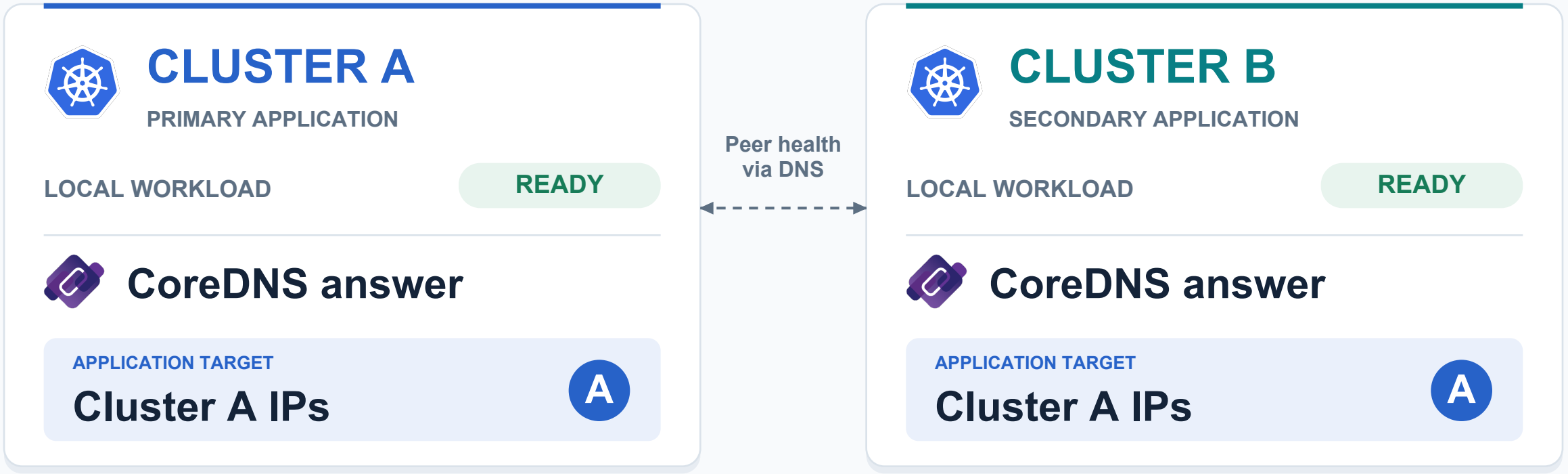


Each k8gb controller queries peer CoreDNS.

Both authorities agree after reconciliation; client caches may lag.

A recovers: the strategy returns to A

A’s application is ready again. The primary preference returns.



Each k8gb controller queries peer CoreDNS.

Both authorities agree after reconciliation; client caches may lag.

Traffic recovery is one part of availability

DNS failover, data recovery, and sessions need separate plans.

k8gb · TRAFFIC STEERING

FAILURE	DNS RESPONSE
APPLICATION Readiness lost	→ **Exclude unready targets**
CLUSTER / REGION Cluster unavailable	→ **Publish healthy cluster targets**

APPLICATION RESPONSIBILITY

STATEFUL DATA

Replication, fencing,
and recovery are yours.

Plan session recovery too.

Traffic recovery ≠ data recovery ≠ session continuity.